PAC-FL: Battery-Efficient & Fair Federated Learning Complete Team Onboarding & Project Handbook

Everything you need to understand: Concept, Architecture, Math, Roles & Month 1 Tasks
Target Outcome: B.Tech Capstone Project + IEEE Conference / Journal Paper (Team of 6)

1. The Big Picture: What is Federated Learning (FL)?

In traditional Machine Learning, all user data (photos, text, sensor logs) is uploaded to a central cloud server like Google or AWS, where a giant model is trained. However, this raises massive privacy concerns and legal issues (GDPR, HIPAA).

Real-World Analogy (Google Gboard)
When your phone predicts the next word you type on your keyboard, Google doesn't send your private chat messages to their servers. Instead, your phone downloads the current model, trains it locally on your typing habits, and sends back only mathematical parameter updates (gradients). The server merges updates from millions of phones. This is Federated Learning!

2. The Real-World Problems We Are Solving

While Federated Learning sounds great in theory, in reality on battery-powered mobile devices, three huge problems occur:

* **Problem 1: Rapid Battery Drain & Mid-Round Dropouts.** Training deep neural networks on edge processors drains phone batteries rapidly. Devices with 15-25% battery frequently shut down or disconnect midway through a training round, wasting communication and time.
* **Problem 2: The 'Unfairness' Trap (Demographic Starvation).** Prior research tried to fix battery drain by simply ignoring low-battery phones ('Greedy Selection'). But in real life, data is Non-IID (people have completely different data). If you only train on high-battery phones, you only train on people with new expensive phones who leave them plugged in! The global model will fail on budget devices.
* **Problem 3: The Mathematical Breakdown (Client Drift).** If low-battery phones do fewer training epochs (e.g. 1 epoch) while high-battery phones do 5 epochs, standard FedAvg averages them incorrectly. The global model drifts toward high-battery clients and fails to converge.

3. Our Solution: The PAC-FL Framework Explained

We are building PAC-FL (Privacy-Preserving Adaptive Controller for Federated Learning). It coordinates 4 smart engineering mechanisms to solve all three problems simultaneously:

The 4 Core Pillars of PAC-FL:

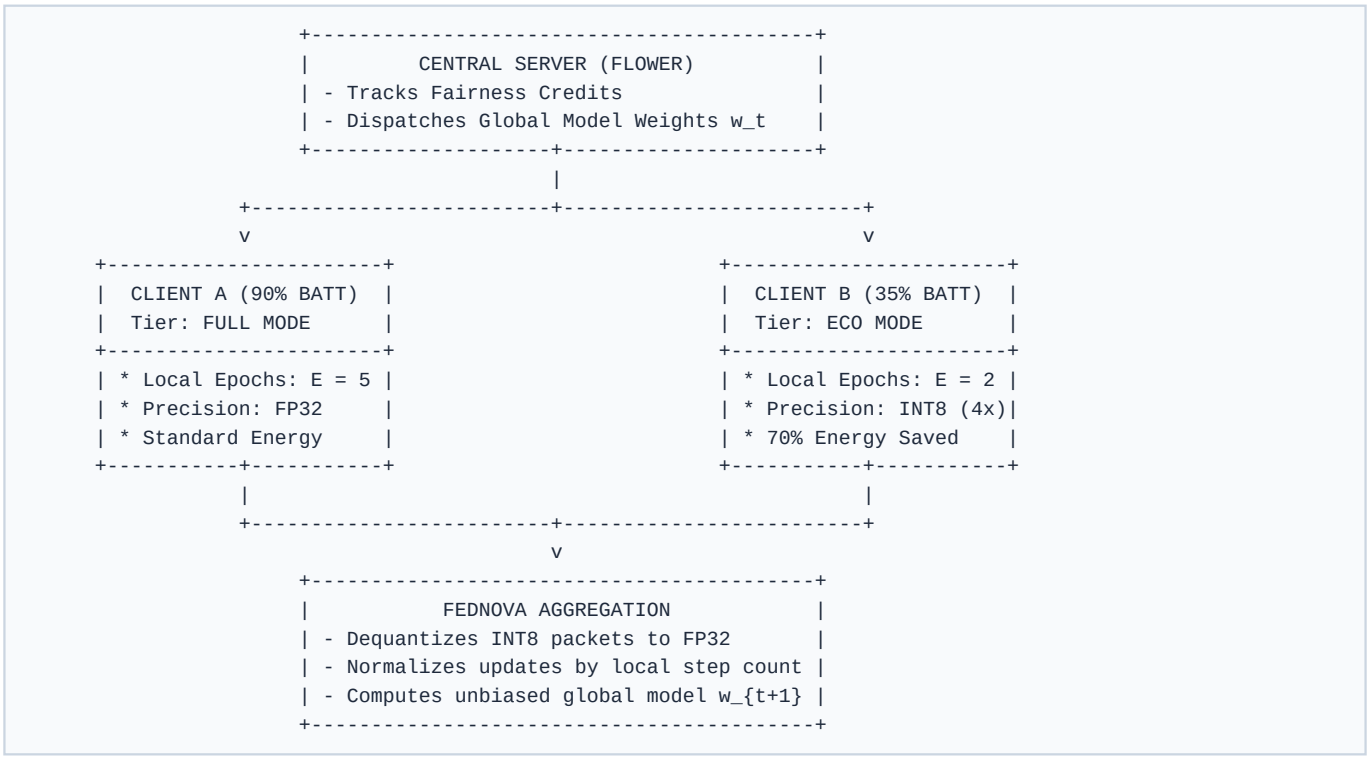
* **1. Smart 3-Tier Battery Governor:** Every round, each device evaluates its battery: >60% -> Full Tier (5 epochs); 30%-60% -> Eco Tier (2 epochs); <30% -> Sleep Tier (skip to save battery).

* **2. INT8 Parameter Quantization (4x Compression):** Phones in Eco Mode compress their weights from 32-bit floats to 8-bit integers before transmitting. This reduces network payload by 75%, slashing wireless radio transmission energy dramatically.

* **3. FedNova Normalized Aggregation (Math Fix):** Instead of naive FedAvg, the server uses FedNova. It divides each device's update by the exact number of training steps it completed, completely eliminating client drift and guaranteeing true mathematical convergence.

* **4. Participation Fairness Credits (Equity Guarantee):** The server tracks how often each client participates. If a low-battery phone is skipped multiple rounds, it earns 'Fairness Credits'. When credits build up, it is granted an emergency slot so its unique data is never lost.

System Flow Diagram (How a Round Works):



4. Team Roles & Individual Ownership (Team of 6)

Each member has a clearly defined responsibility, a dedicated code module to build, and a specific section to author in our final IEEE conference/journal paper. No overlapping confusion!

Member 1: Tech Lead & FL Architect

Main Job: Set up the project repository, configure Python virtual environments, and build the Flower (flwr) simulation engine that coordinates clients and server.
Code Files: src/server.py, src/client.py, run_simulation.py
Paper Section: Section III - System Architecture & Framework Implementation.

Member 2: Aggregation & Optimization Specialist (FedNova)

Main Job: Implement the custom Flower Strategy that replaces standard FedAvg with FedNova. Mathematically normalizes updates so varying epochs don't break training.
Code Files: src/strategy/fednova_strategy.py
Paper Section: Section IV - Mathematical Formulation & Objective Consistency.

Member 3: Compression & Quantization Engineer

Main Job: Build the INT8 tensor compression functions (FP32 <-> INT8) in PyTorch. Measure exact megabytes saved per round and verify that accuracy drop is negligible (<0.5%).
Code Files: src/compression/quantizer.py
Paper Section: Section V - Communication Compression & Quantization Overhead.

Member 4: Energy & Battery Modeling Specialist

Main Job: Formulate the physical energy model for compute (FLOPs x clock speed) and wireless transmission. Build the battery state simulator that decrements battery per epoch and upload.
Code Files: src/energy/battery_simulator.py, src/energy/energy_model.py
Paper Section: Section VI - Edge Energy Modeling & Battery Discharge Simulation.

Member 5: Adaptive Controller & Fairness Lead

Main Job: Build the 3-tier battery governor logic (Full/Eco/Sleep) and implement the Fairness Credit counter. Calculate Jain's Fairness Index to prove our method treats devices equitably.
Code Files: src/controller/battery_governor.py, src/fairness/fairness_tracker.py
Paper Section: Section VII - Adaptive Resource Governor & Representation Fairness.

Member 6: Data Pipeline, Benchmarks & Paper Lead

Main Job: Download CIFAR-10, implement the Dirichlet non-IID partitioner (alpha=0.5), coordinate running the 4 experimental baselines, and manage the IEEE Overleaf LaTeX paper template.
Code Files: src/data/cifar10_partitioner.py, benchmarks/run_baselines.py
Paper Section: Abstract, Section I (Intro), Section VIII (Experimental Evaluation).

5. Month 1 (Weeks 1 to 4): Step-by-Step Action Plan

During Month 1, every member works independently on their module. You do NOT need to wait for others!

Week 1: Environment & Foundations (Oct 7 - Oct 14)

* **All Members:** Install Python 3.11, PyTorch, Torchvision, and Flower (`pip install flwr[simulation]`). Clone repo.
* **Member 1 & Member 6:** Initialize folder structure and build simple 3-layer CNN on CIFAR-10.

Week 2: Data Split & Battery Simulator (Oct 15 - Oct 21)

* **Member 6:** Implement Dirichlet partitioner (`alpha = 0.5`). Verify that client 0 has different class distributions than client 1.
* **Member 4:** Write `battery_simulator.py`. Assign initial batteries (15% to 100%) to 20 clients.
* **Member 1:** Run basic vanilla FedAvg on CIFAR-10 using Flower simulation.

Week 3: Algorithm Prototypes (Oct 22 - Oct 28)

* **Member 3:** Write `quantize_tensor(t)` and `dequantize_tensor(q, scale)` in PyTorch. Verify output matches original tensor.
* **Member 2:** Write `FedNovaStrategy` skeleton in Flower that accepts client epoch counts.
* **Member 5:** Write rule conditions: if battery > 60% return Full, if 30-60% return Eco, else Sleep.

Week 4: Unit Testing & Milestone 1 (Oct 29 - Nov 5)

* **Member 5:** Add Fairness Credit counter. If client is skipped 3 times, grant credit.
* **All Members:** Run standalone tests for each module. Milestone 1 Complete: All 6 pieces ready for integration in Month 2!

6. Quick Glossary: Plain-English Definitions

* **Non-IID Data:** Non-Independent and Identically Distributed. Means each user's phone has different data (e.g., Client A only has pictures of dogs, Client B only has pictures of planes).
* **Local Epoch (E):** How many times a phone passes through its own local data before sending updates back.
* **INT8 Quantization:** Compressing 32-bit floating point numbers into 8-bit integers. Reduces file size by 75%.
* **Client Drift:** When different clients train for different amounts of time, the global model gets pulled toward the faster clients and drifts away from the correct answer.
* **FedNova:** A special mathematical algorithm created by researchers at NeurIPS that fixes Client Drift by normalizing update steps.
* **Jain's Fairness Index:** A mathematical score between 0 and 1.0 that measures fairness. 1.0 means every phone participated equally.

Need Help or Clarification?
Don't worry if you don't know everything on Day 1! We are working as a team. Member 1 (Tech Lead) and your teammates are here to help. Reach out in the WhatsApp group whenever you get stuck!